Testes Lógicos & NA's

Mais sobre filtros e o começo de visualizações

James Sampaio, PhD

Filtros Lógicos & NA’s

Mais sobre filtros e valores faltantes

- Testes lógicos
- Valores faltantes
- Diamonds data frame

Testes Lógicos

Comparando

O que irão retornar os comandos abaixo ?

```
1 < 3
[1] TRUE

1 > 3
[1] FALSE

c(1, 2, 3, 4, 5) > 3
[1] FALSE FALSE FALSE  TRUE  TRUE
```

SUA VEZ

```
x <- c(1, 2, 3, 4, 5)
```

Operador	Resultado	Comparação
`x > 3`	`F F F T T`	maior que
`x >= 3`	`F F T T T`	maior ou igual a
`x < 3`	`T T F F F`	menor que
`x <= 3`	`T T T F F`	menor ou igual a
`x == 3`	`F F T F F`	igual a
`x != 3`	`T T F T T`	diferente de
`x = 3`		mesmo que <-

%in%

O que irão retornar os comandos abaixo ?

```
1 %in% c(1, 2, 3, 4)
[1] TRUE

1 %in% c(2, 3, 4)
[1] FALSE

c(3, 4, 5, 6) %in% c(2, 3, 4)
[1] TRUE TRUE FALSE FALSE
```

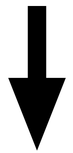
Operadores Booleanos

Combine testes lógicos por meio de `&`**,** `|` **,** `xor`**,** `!`**,** `any` **e** `all`

```
7 > 2  &  7 < 9
```



```
TRUE  &  TRUE
```



```
TRUE
```

&

Tanto a condição 1 como a condição 2 devem ser verdadeiras

Expressão	Resultado
`TRUE & TRUE`	`TRUE`
`TRUE & FALSE`	`FALSE`
`FALSE & TRUE`	`FALSE`
`FALSE & FALSE`	`FALSE`

Ao menos umas das condições 1 e 2 deve ser verdadeira

Expressão	Resultado
`TRUE | TRUE`	`TRUE`
`TRUE | FALSE`	`TRUE`
`FALSE | TRUE`	`TRUE`
`FALSE | FALSE`	`FALSE`

xor

Ao menos umas das condições 1 e 2 deve ser verdadeira, mas não ambas

Expressão	Resultado
`xor(TRUE, TRUE)`	`FALSE`
`xor(TRUE, FALSE)`	`TRUE`
`xor(FALSE, TRUE)`	`TRUE`
`xor(FALSE, FALSE)`	`FALSE`

!

Negação

Expressão	Resultado
`!(TRUE)`	`FALSE`
`!(FALSE)`	`TRUE`

any

Ao menos uma entre as condições é verdadeira

Expressão	Resultado
`any( c(TRUE, FALSE, FALSE) )`	`TRUE`
`any( c(FALSE, FALSE, FALSE) )`	`FALSE`

all

Todas as condições são verdadeiras

Expressão	Resultado
`all( c(TRUE, TRUE, TRUE) )`	`TRUE`
`all( c(TRUE, FALSE, TRUE) )`	`FALSE`

SUA VEZ

Transforme a sentença abaixo em uma questão lógica no R

```
w <- c(-1, 0, 1)
```

w é positivo?

```
w > 0
```

```
[1] FALSE FALSE TRUE
```

SUA VEZ

Transforme a sentença abaixo em uma questão lógica no R

```
x <- c(5,15)
```

`x` é maior que 10 e menor que 20?

```
x > 10 & x < 20
[1] FALSE TRUE
```

SUA VEZ

Transforme a sentença abaixo em uma questão lógica no R

```
y <- "Fevereiro"
```

O objeto `y` é a palavra Fevereiro?

```
y == "Fevereiro"
[1] TRUE
```

SUA VEZ

Transforme a questão abaixo em uma sentença lógica no R

```
z <- c("Segunda", "Quinta", "Sexta")
```

Todo elemento em `z` é um dia da semana?

```
all( z %in% c("Domingo", "Segunda", "Terça",
 "Quarta", "Quinta", "Sexta", "Sábado") )
```

```
[1] TRUE
```

Erros comuns

```
x > 10 & < 20

y = "Fevereiro"

all( z == "Domingo" | "Segunda" | "Terça" … )
```

Valores Faltantes

Dados faltantes

Um problema comum

- Na vida real lidaremos com muitos problemas na obtenção de dados
- Um dos problemas são os dados faltantes. Eles podem ocorrer por diversas razões
- No R, os valores faltantes são indicados por `NA`

Dados faltantes

Um problema comum

Expressão	Resultado
`5 + NA`	`NA`
`mean( c(5, NA) )`	`NA`
`NA < 3`	`NA`
`NA == 3`	`NA`
`NA == NA`	`NA`

Dados faltantes

Comportamento

- Valores faltantes se propagam

```
a <- c(1, NA)
a == NA
[1] NA NA
```

- Utilize `is.na()` para checar possíveis valores faltantes

```
is.na(a)
[1] FALSE TRUE
```

na.rm

- Muitas funções permitem ignorar valores faltantes antes do cômputo a partir do argumento `na.rm`

```
b <- c(1, 2, 3, 4, NA)

sum(b)

[1] NA


sum(b, na.rm = TRUE)

[1] 10
```

na.rm

- Muitas funções permitem ignorar valores faltantes antes do cômputo a partir do argumento `na.rm`

```
b <- c(1, 2, 3, 4, NA)
mean(b)
[1] NA


mean(b, na.rm = TRUE)
[1] 2.5
```

Diamonds

Um famoso conjunto de dados do tidyverse

Diamonds

Um famoso conjunto de dados do tidyverse

- ~**54.000** diamantes lapidados obtidos do site http://www.diamondse.info/
- Disponível no pacote ggplot2
- quilate (*carat*), corte *(cut)*, cor *(color)* e clareza *(clarity)*
- profundidade total *(depth)*, topo *(table)* e *preço (price)*
- comprimento *(x)*, largura *(y)* e *profundidade (z)*

Vista de lado

x

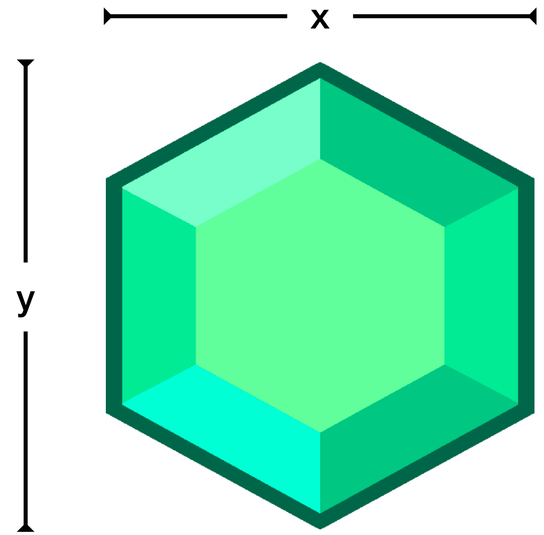
table width

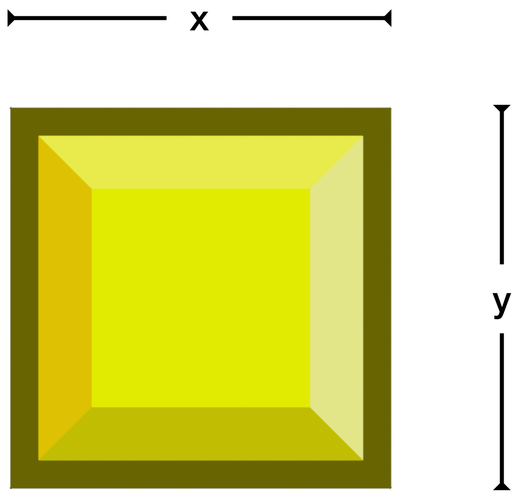


z

table = 100 * (table width / x)

Vista de cima





depth = 100 * (z / mean(x, y))

SUA VEZ

Avalie as seis primeiras linhas do banco diamonds

```
> diamonds[1:6, ]
# A tibble: 6 × 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
```

SUA VEZ

Avalie as seis primeiras linhas do banco diamonds

```
> head(diamonds)
# A tibble: 6 × 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
```

SUA VEZ

Avalie as seis últimas linhas do banco diamonds

```
> nrow(diamonds)
[1] 53940
> diamonds[53935:53940, ]
# A tibble: 6 × 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.72 Premium   D     SI1      62.7    59  2757  5.69  5.73  3.58
2  0.72 Ideal     D     SI1      60.8    57  2757  5.75  5.76  3.5
3  0.72 Good      D     SI1      63.1    55  2757  5.69  5.75  3.61
4  0.7  Very Good D     SI1      62.8    60  2757  5.66  5.68  3.56
5  0.86 Premium   H     SI2      61      58  2757  6.15  6.12  3.74
6  0.75 Ideal     D     SI2      62.2    55  2757  5.83  5.87  3.64
```

SUA VEZ

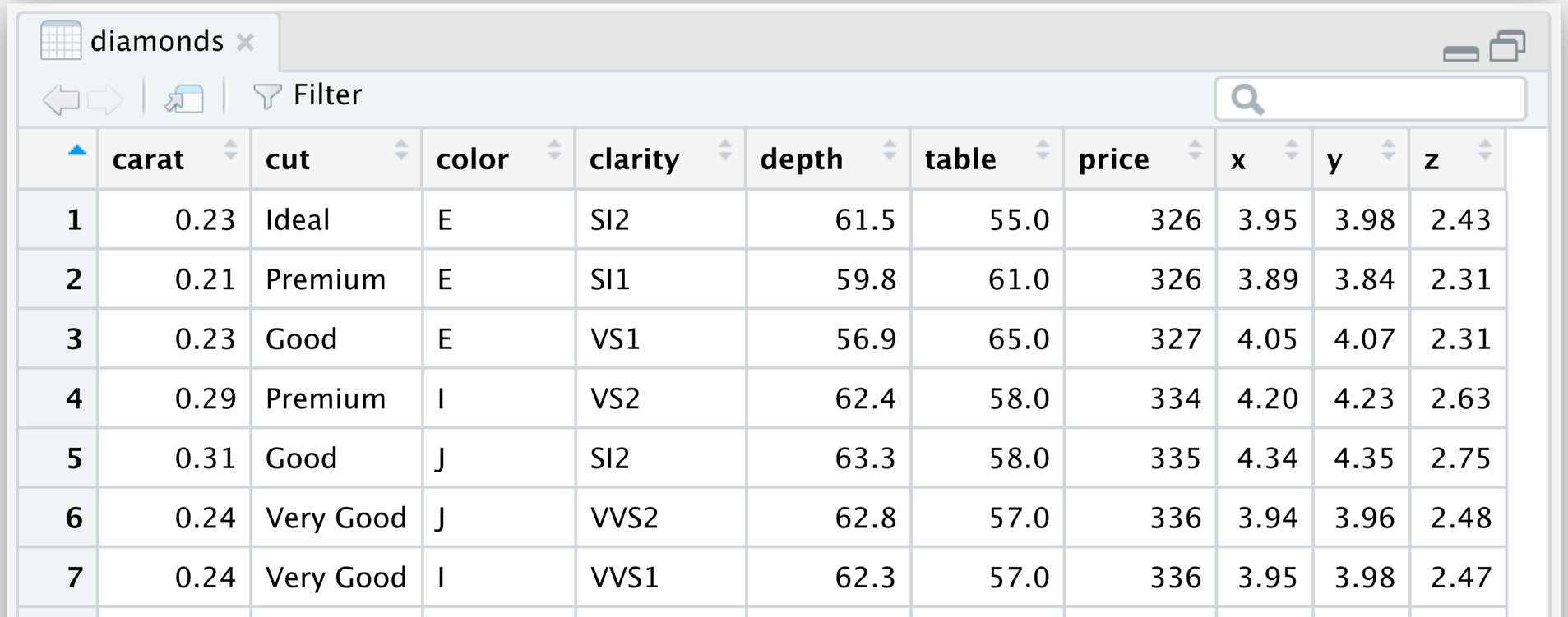
Avalie as seis últimas linhas do banco diamonds

```
> tail(diamonds)
# A tibble: 6 × 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.72 Premium   D     SI1      62.7    59  2757  5.69  5.73  3.58
2  0.72 Ideal     D     SI1      60.8    57  2757  5.75  5.76  3.5
3  0.72 Good      D     SI1      63.1    55  2757  5.69  5.75  3.61
4  0.7  Very Good D     SI1      62.8    60  2757  5.66  5.68  3.56
5  0.86 Premium   H     SI2      61      58  2757  6.15  6.12  3.74
6  0.75 Ideal     D     SI2      62.2    55  2757  5.83  5.87  3.64
```

View

Por meio do comando `View` você poderá visualizar os dados em uma janela

```
View(diamonds)
```



diamonds

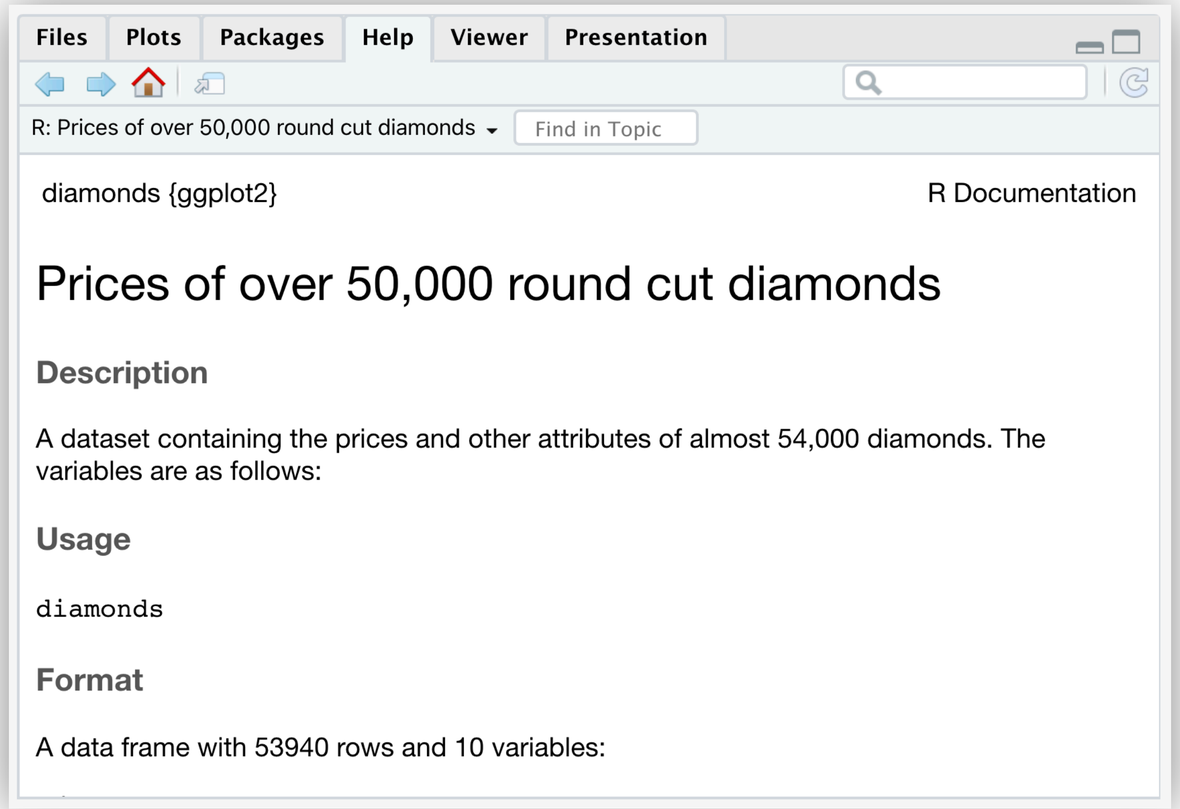
Filter

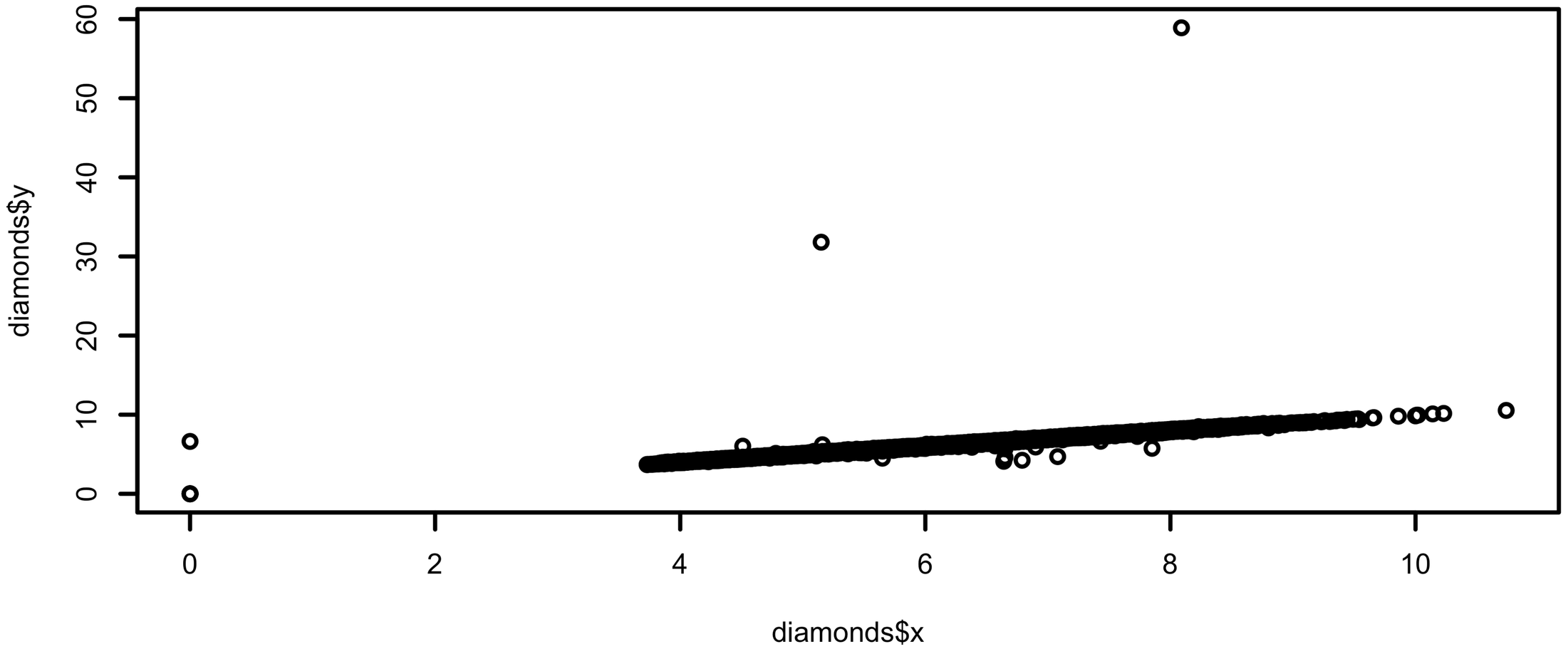
	carat	cut	color	clarity	depth	table	price	x	y	z
1	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
2	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
3	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
4	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
5	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
6	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
7	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47

Help

Você pode acessar ajuda sobre qualquer comando do R por meio da "?"

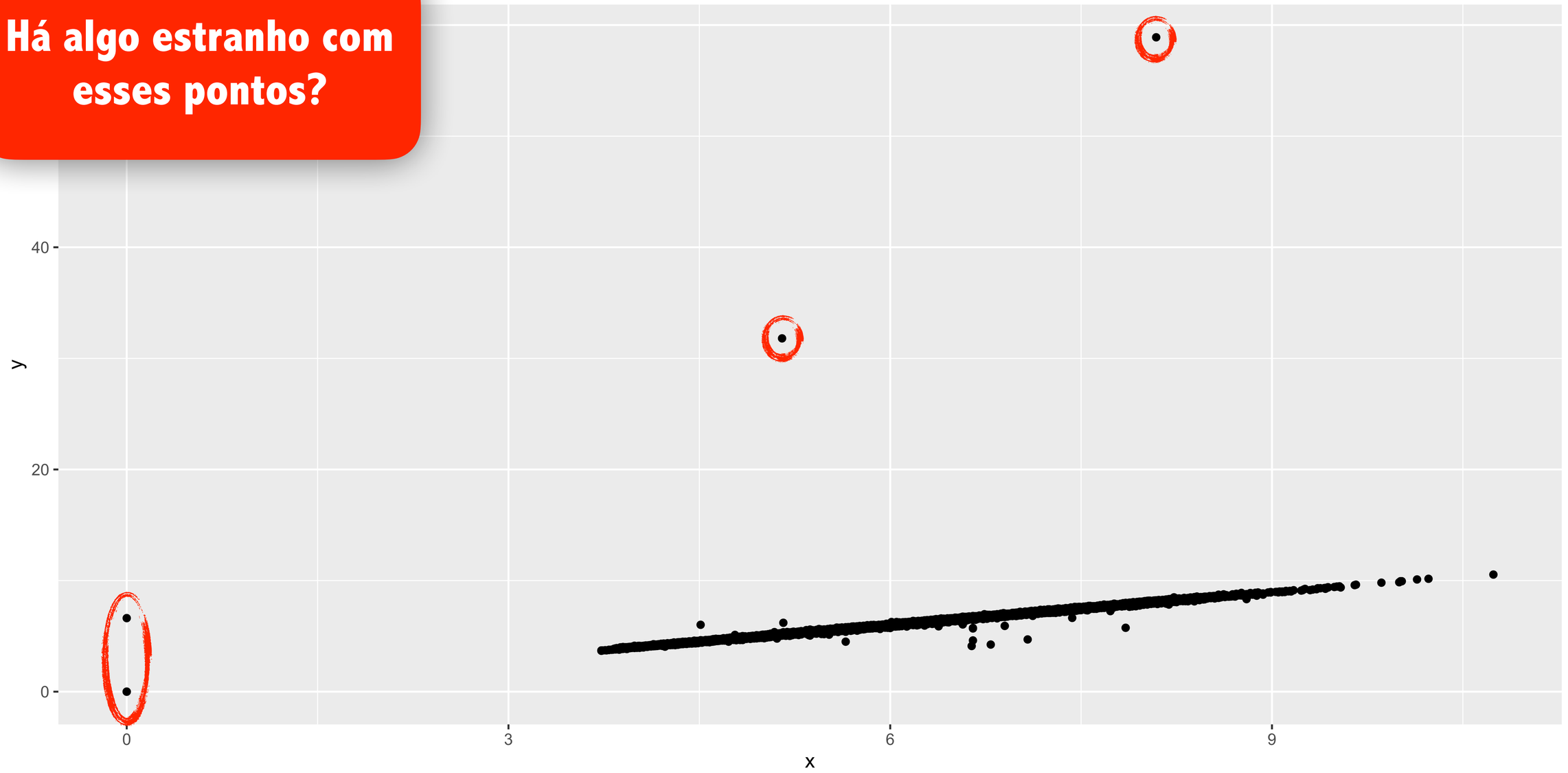
```
?diamonds
```






```
plot(diamonds$x, diamonds$y)
```

Há algo estranho com esses pontos?



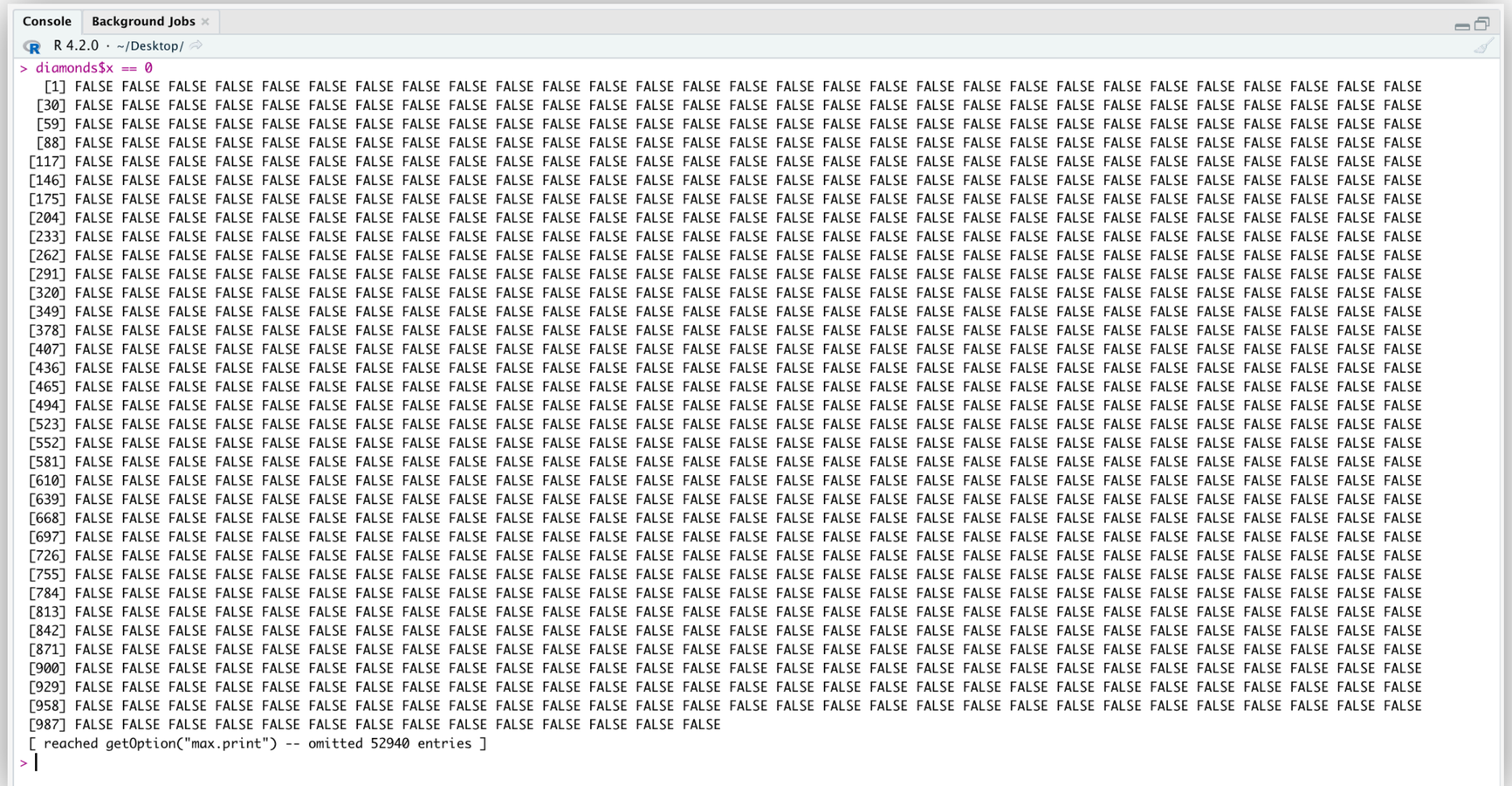

```
qplot(x, y, data = diamonds)
```

Filtros lógicos para diamonds

- Combinar testes lógicos e filtragem é uma técnica muito poderosa

```
diamonds$x == 0
```

?



Filtros lógicos para diamonds

- Combinar testes lógicos e filtragem é uma técnica muito poderosa

```
x_zeroes <- diamonds$x == 0

diamonds[x_zeroes, ]
```

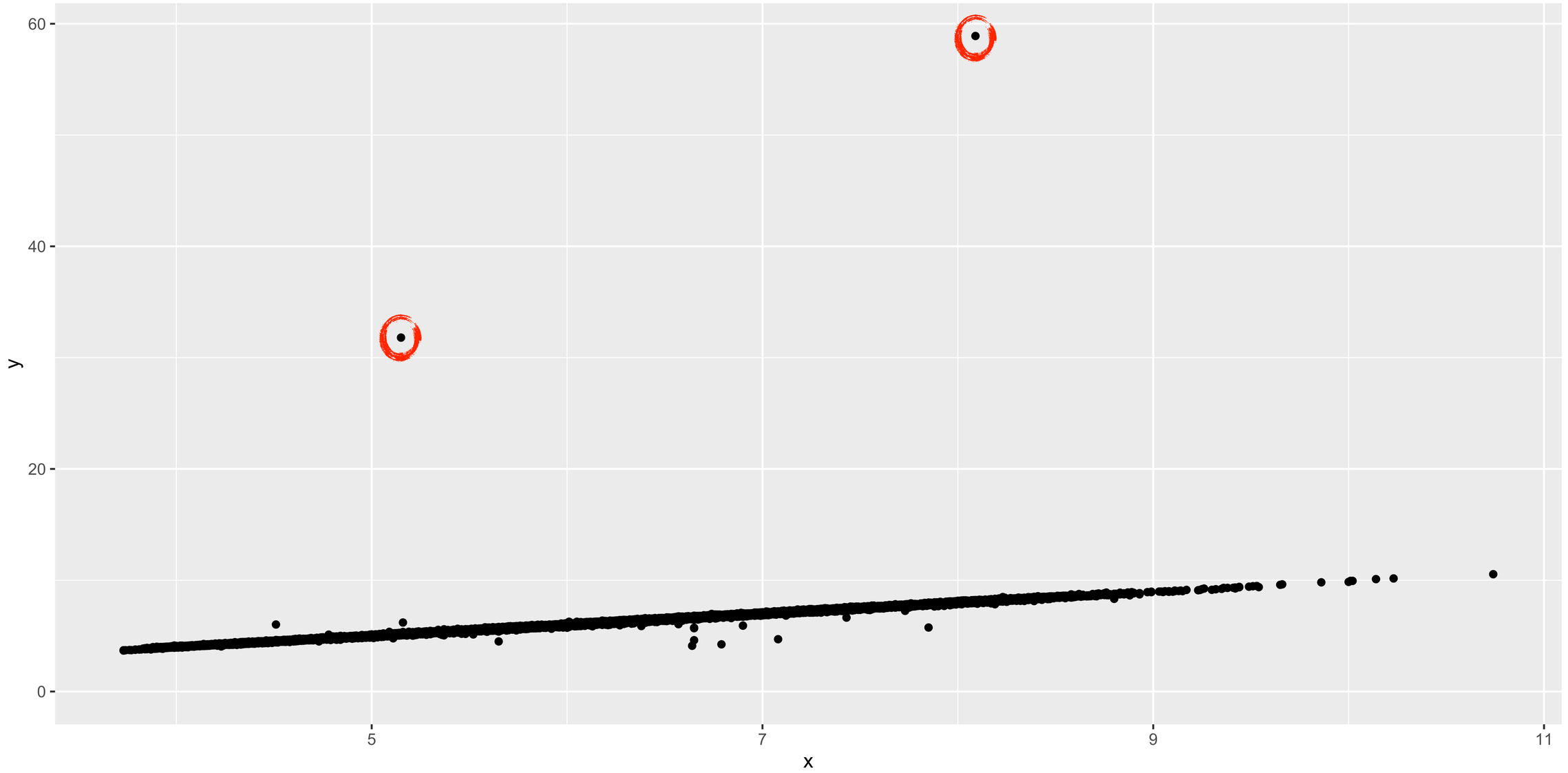
```
> x_zeroes <- diamonds$x == 0
> diamonds[x_zeroes, ]
# A tibble: 8 × 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  1.07 Ideal     F     SI2      61.6    56  4954     0  6.62     0
2  1    Very Good H     VS2      63.3    53  5139     0  0        0
3  1.14 Fair      G     VS1      57.5    67  6381     0  0        0
4  1.56 Ideal     G     VS2      62.2    54 12800     0  0        0
5  1.2  Premium   D     VVS1     62.1    59 15686     0  0        0
6  2.25 Premium   H     SI2      62.8    59 18034     0  0        0
7  0.71 Good      F     SI2      64.1    60  2130     0  0        0
8  0.71 Good      F     SI2      64.1    60  2130     0  0        0
>
```

Filtros lógicos para diamonds

- Combinar testes lógicos e filtragem é uma técnica muito poderosa

```
x_zeroes <- diamonds$x == 0
diamonds$x[x_zeroes] <- NA
```

O que irá acontecer com o gráfico?




```
qplot(x, y, data = diamonds)
```

Filtros lógicos para diamonds

- Combinar testes lógicos e filtragem é uma técnica muito poderosa

```
y_maior_20 <- diamonds$y > 20

diamonds[y_maior_20, ]
```

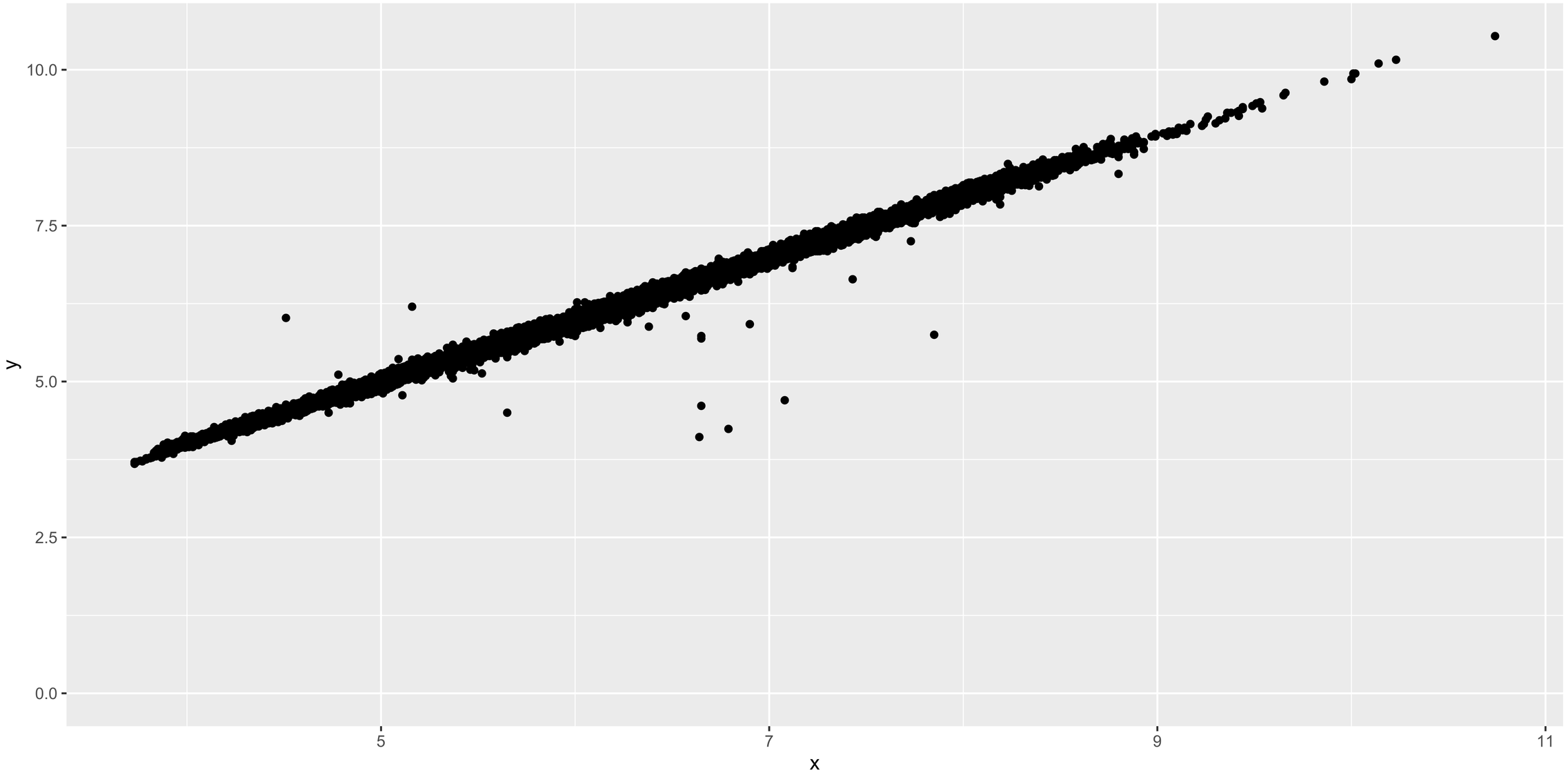
```
> y_maior_20 <- diamonds$y > 20
> diamonds[y_maior_20, ]
# A tibble: 2 × 10
  carat cut   color clarity depth table price     x     y     z
  <dbl> <ord> <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  2    Prem… H     SI2      58.9    57 12210  8.09  58.9  8.06
2  0.51 Ideal E     VS1      61.8    55  2075  5.15  31.8  5.12
```

Filtros lógicos para diamonds

- Combinar testes lógicos e filtragem é uma técnica muito poderosa

```
y_maior_20 <- diamonds$y > 20
diamonds$y[y_maior_20] <- NA
```

O que irá acontecer com o gráfico?




```
qplot(x, y, data = diamonds)
```

Segurança

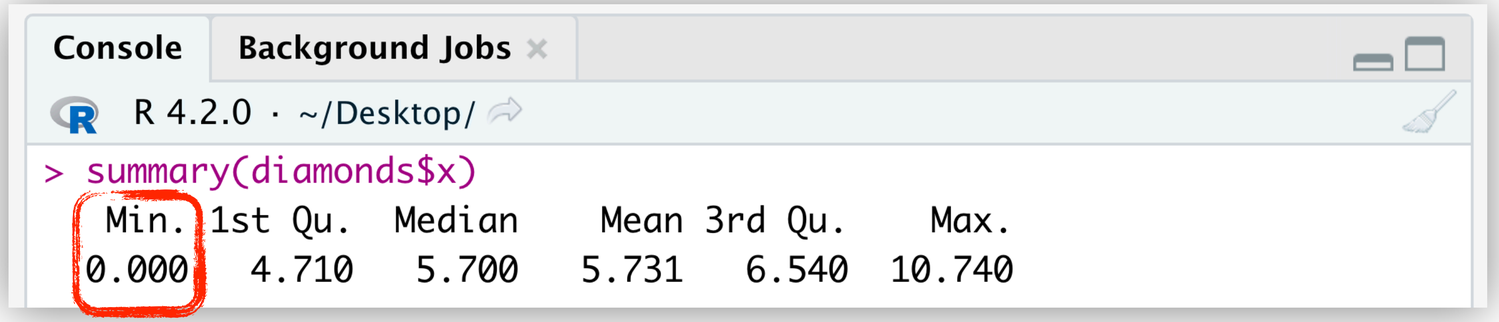
- O que acabamos de fazer não é a melhor forma, pois alteramos definitivamente o conjunto de dados original

- Como o conjunto de dados diamonds pertence ao ggplot2 podemos recuperá-lo facilmente

- Basta removermos o conjunto por meio do comando `rm` e carregá-lo novamente

```
rm(diamonds)

summary(diamonds$x)
```



Segurança

- A melhor forma de evitarmos esse problema é salvar uma cópia do conjunto de dados em um novo objeto

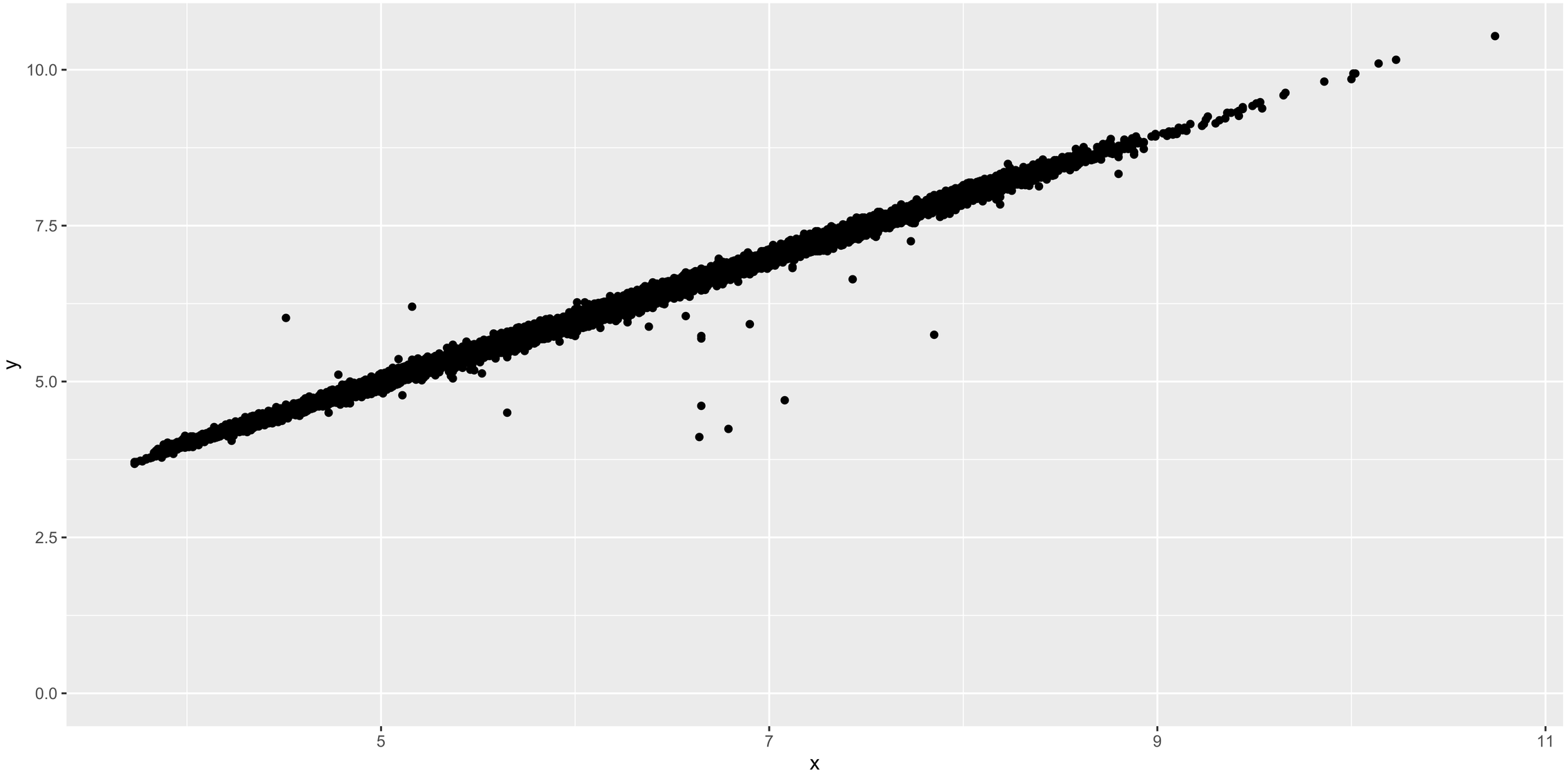
```
diamonds_NA <- diamonds

x_zeroes <- diamonds_NA$x == 0

diamonds_NA$x[x_zeroes] <- NA

y_maior_20 <- diamonds_NA$y > 20

diamonds_NA$y[y_maior_20] <- NA
```




```
qplot(x, y, data = diamonds_NA)
```